

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering

ASSESSMENT TOOL 2 - FLOWCHART-TO-CODE CONVERSION

Course Code:	CSA0305	Course Title:	Data Structures
CO Assessed:	CO5 - Naturalization (BL4, BL6)	Assessment:	Assessment Tool 2 - Flowchart-To-Code Conversion
Weightage:	35% of CIA	Total Marks:	100
CO5 Statement:	<i>Construct MST using shortest path algorithms and traverse the graph to model and analyse real-world problem.</i>		
Student Name:		Reg. No.:	
Section / Batch:		Date:	04/09/2026

Code of Conduct

I _____ (Name / Reg No)

certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Instructions

- This test contains 5 Flowchart-to-code conversion questions. Total: 100 Marks.
- When a student answer using Flowchart-to-code conversion should evaluate based on the ability to design clear, logical, and efficient code solutions for data structure problems.
- No negative marking. Unanswered questions carry zero marks.

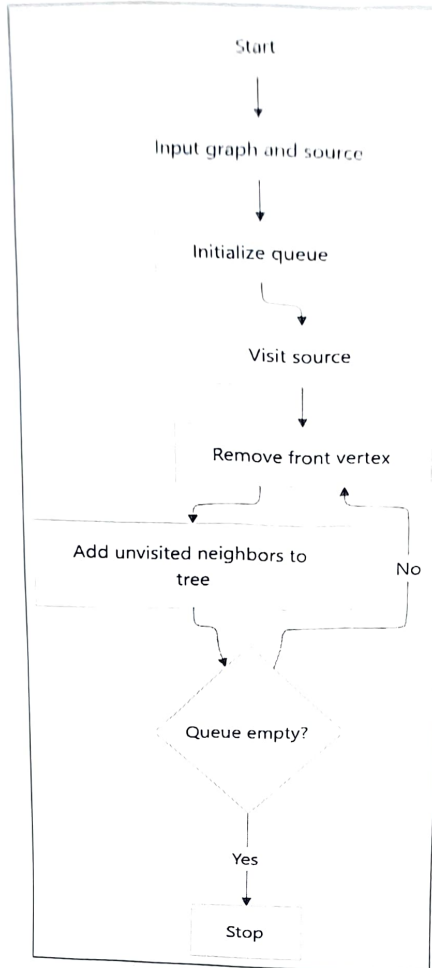
Section / Topic	Focus	Q Nos	Marks
Graph Traversal	BFS	1	20
Shortest Path Algorithm	Dijkstra's Algorithm	2	20
Shortest Path Algorithm	Unweighted Graph	3	20
Graph Traversal	BFS	4	20
Minimum Spanning Tree	Prim's Algorithm	5	20
GRAND TOTAL:			100 Marks

Annexure A - Pseudocode Writing Task

1. Convert the flowchart for generating a BFS tree into code.

(20 Marks)

Flowchart Diagram:



2. Convert the flowchart for Dijkstra's shortest path algorithm into code.

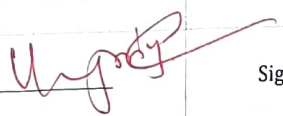
Flowchart Diagram:

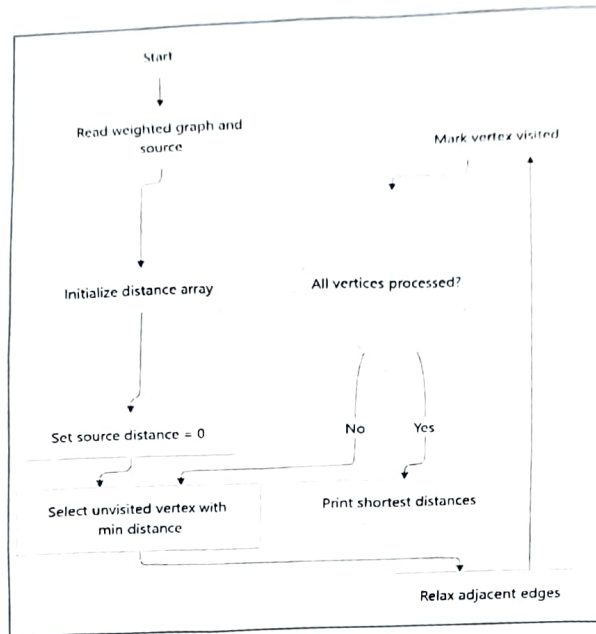
(20 Marks)

Total Minimum spanning

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20	Demonstrates complete and clear understanding; handles edge cases effectively	Good understanding; minor gaps in edge case handling	Partial understanding; misses some requirements	Misinterprets problem; major gaps
Code Correctness	30	Fully correct; passes all test cases	Mostly correct; minor errors	Partially correct; several test cases fail	Incorrect or non-functional code
Code Quality & Readability	20	Clean, modular, well-commented, follows standards	Readable with some structure	Limited readability; poor structure	Unorganized and hard to read
Complexity Analysis	20	Accurate time & space complexity with justification	Correct but limited explanation	Incomplete or partially correct	Missing or incorrect
Testing & Validation	10	Thorough testing including edge cases	Adequate testing with few edge cases	Minimal testing	No testing evidence

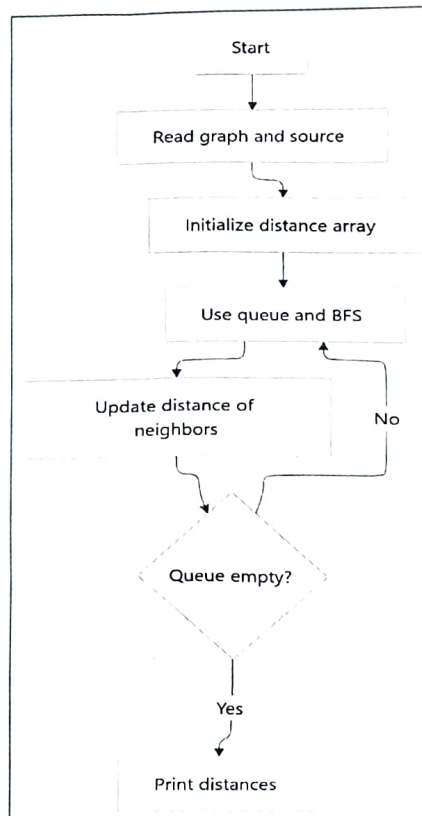
Faculty In-charge	Course Coordinator	Program Director
Signature: 	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____



3. Convert the flowchart for shortest path in an unweighted graph into code.

(20 Marks)

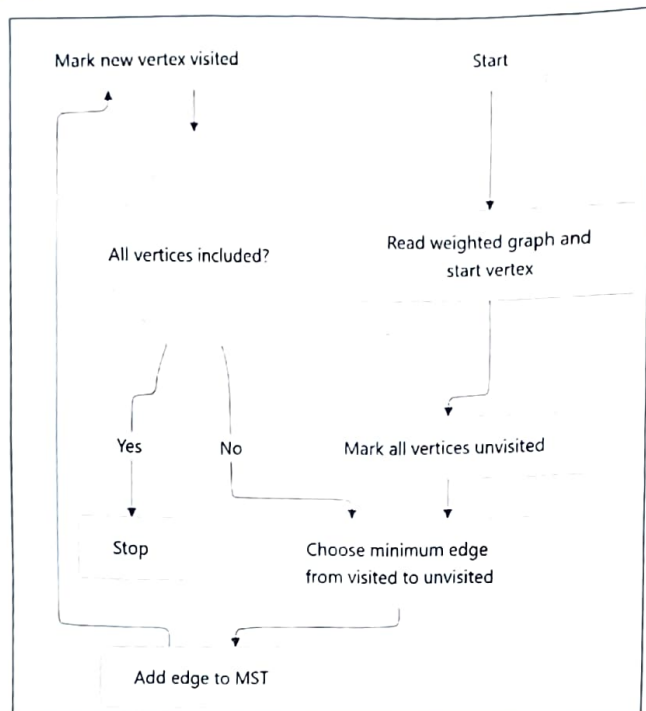
Flowchart Diagram:



4. Convert the flowchart for Prim's algorithm into code.

(20 Marks)

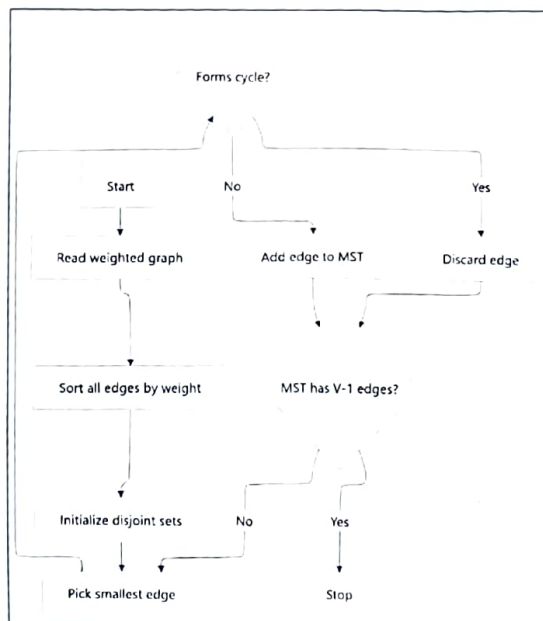
Flowchart Diagram:



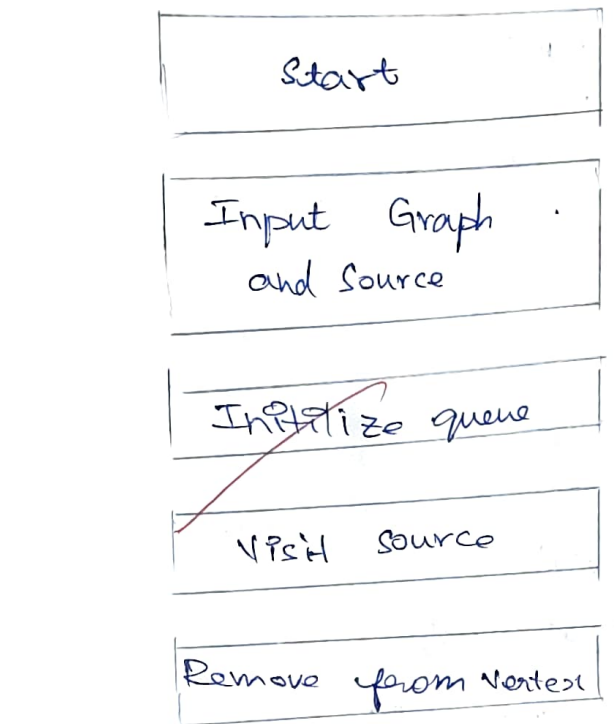
5. Convert the flowchart for Kruskal's algorithm into code.

(20 Marks)

Flowchart Diagram:

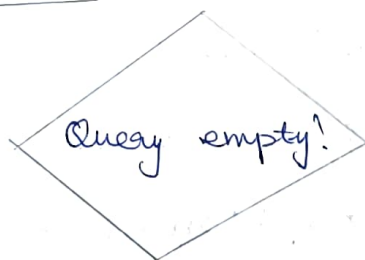


① Convert the flowchart for generating BFS tree into code.



And unvisited
neighbours to
tree

No



Yes

stop

③

```
int main()
```

```
{ int start;
```

```
printf("Enter number of vertices: ");
```

```
scanf("%d", &n);
```

```
printf("Enter adjacency matrix: \n");
```

```
for (int i=0; i<n; i++)
```

```
{
```

```
for (int j=0; j<n; j++)
```

```
{ scanf("%d", &graph[i][j]);
```

```
}
```

```
printf("Enter starting vertex: ");
```

```
scanf("%d", &start);
```

```
BFS(start);
```

```
return 0;
```

```
}
```

Output

Enter the number of vertices: 4

Enter the adjacency matrix

0 1 1 0

1 0 0 1

1 0 0 1

0 1 1 0

Enter starting vertex: 0

BFS Traversal: 0

BFS Tree Edge: 0 → 1

BFS Tree Edge: 0 → 2

BFS Tree Edge: 1 → 3

② Dijkstra's Shortest Path Algorithm

```
#include <stdio.h>
#include <limits.h>

#define MAX 20
int graph [MAX][MAX];
int distanc [MAX];
int visited [MAX];

int n
/* Find the unvisited vertex with minimum distance */

int findMinVertex()
{
    int min = INT_MAX;
    int minVertex = -1;

    for (int i = 0; i < n; i++)
    {
        if (visited[i] == 0 && distanc[i] < min)
        {
            min = distanc[i];
            minVertex = i;
        }
    }

    return minVertex;
}
```

/* Dijkstra's Algorithm */

void dijkstra (int source)

```
{
    /* Initialization */
```

for (int i=0; i<n; i++)

```
{
    distance[i] = INT_MAX;
    visited[i] = 0;
}
```

distance[source] = 0;

/* Process all vertices */

for (int count=0; count<n-1; count++)

```
{
    int u = findMIN Vertex();
    if (u == -1)
        break;
    visited[u] = 1;
```

/* Relax adjacent vertices */

for (int v=0; v<n; v++)

```
{ if (visited[v] == 0 && graph[u][v] > 0
```

```
distance[u] < INT_MAX &&
```

```
distance[u] + graph[u][v] < distance[v])
```

```
{ distance[v] = distance[u] + graph[u][v];
```

```
}
```

```
}
```

/* Display shortest distances */

```
printf("\n Shortest distances from vertex %d: \n", source);
```

for (int i=0; i<n; i++)

```
{ if (distance[i] == INT_MAX)
```

```

{   print ("Vertex %d: INT \n", i),
}

else
{   printf ("Vertex %d: %d \n", i, distance[i]),
}

}

int main()
{   int source;
    printf ("Enter number of vertices: "); scanf ("%d", &n);
    printf ("Enter weighted adjacency matrix: \n");
    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j < n; j++)
        {   scanf ("%d", &graph[i][j]),
        }
    }
    printf ("Enter source vertex: ");
    scanf ("%d", &source);
    dijkstra (source);
    return 0;
}

```

Output

Enter number of vertices: 5

Enter weighted adjacency matrix:

0	10	0	30	100
10	0	50	0	0
0	50	0	20	10
30	0	20	0	60
100	0	10	60	0

Enter source vertex: 0

Shortest distance from vertex 0:

Vertex 0: 0, Vertex 4: 60
 Vertex 1: 10,
 Vertex 2: 50,
 Vertex 3: 30

Start

Read Weighted graph
and source

Traverse graph
using

DFS

DFS

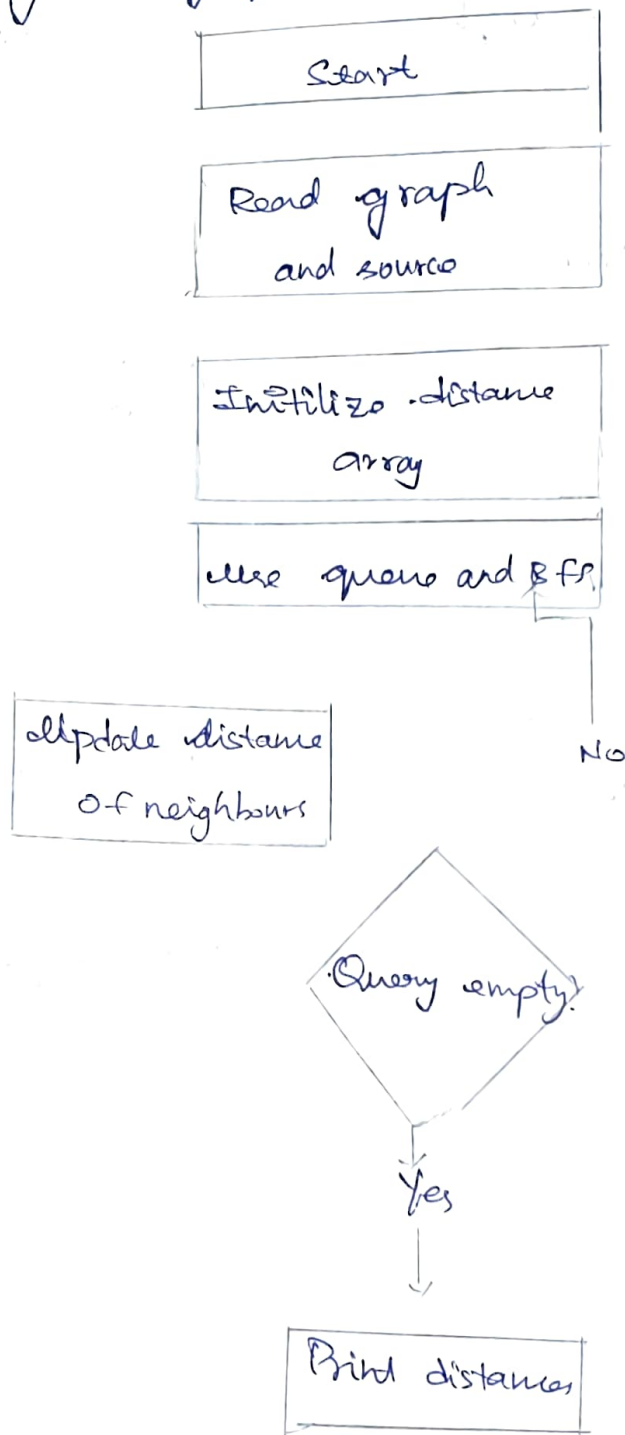
DFS

DFS

DFS

DFS

③ Convert the flowchart for shortest path in an unweighted graph into code.



#include <stdio.h>

#define MAX 20

int graph [MAX][MAX];

int visited [MAX];

int distance [MAX];

int parent [MAX];

int queue [MAX];

int n;

int front = 0;

int rear = 0;

/* BFS for shortest path */

void BFS (int source)

{
 /* Initialize */

 for (int i = 0; i < n; i++)

 {
 visited[i] = 0;

 distance[i] = -1;

 parent[i] = -1;

 }

 /* Start from source */

 visited[source] = 1;

 distance[source] = 0;

 queue[rear++] = source;

 /* BFS traversal */

 while (front < rear)

 {
 int current = queue[front++];

```
for (int i = 0; i < n; i++)
```

```
{ if (graph[current][i] != 1 &&  
    visited[i] == 0)
```

```
{ visited[i] = 1;
```

```
  distance[i] = 1
```

```
  distance[current] + 1)
```

```
  parent[i] = current;
```

```
  queue[rear++] = i;
```

```
}
```

```
}
```

```
}
```

```
// Print Shortest Path */
```

```
void printPath(int vertex)
```

```
{ if (vertex == -1)
```

```
    return;
```

```
    printPath(parent[vertex]);
```

```
    printf("%d ", vertex);
```

```
}
```

```
int main()
```

```
{ int source;
```

```
  int destination;
```

```
  printf("Enter number of vertices: ");
```

```
  scanf("%d", &n);
```

```
  printf("Enter adjacency matrix: \n");
```

```
  for (int i = 0; i < n; i++)
```

```
  { for (int j = 0; j < n; j++)
```


{ scanf("%d", & graph[i][j])
}
}

printf("Enter source vertex: ");
scanf("%d", & source);

BFS (source);

printf("Enter destination vertex: ");
scanf("%d", & destination);

if (distance [destination] == -1)

{ printf("\n No path exists from %d to %d.\n",
source, destination);

}

else

{ printf("\n Shortest distance = %d\n",
distance [destination]);

printf("Shortest path: ");

PrintPath (destination);

printf("\n");

}

return 0;

}

Input

Enter number of vertices: 5

Enter adjacency matrix:

0	1	1	0	0
1	0	0	1	0
1	0	0	1	0
0	1	1	0	0
0	0	0	1	0

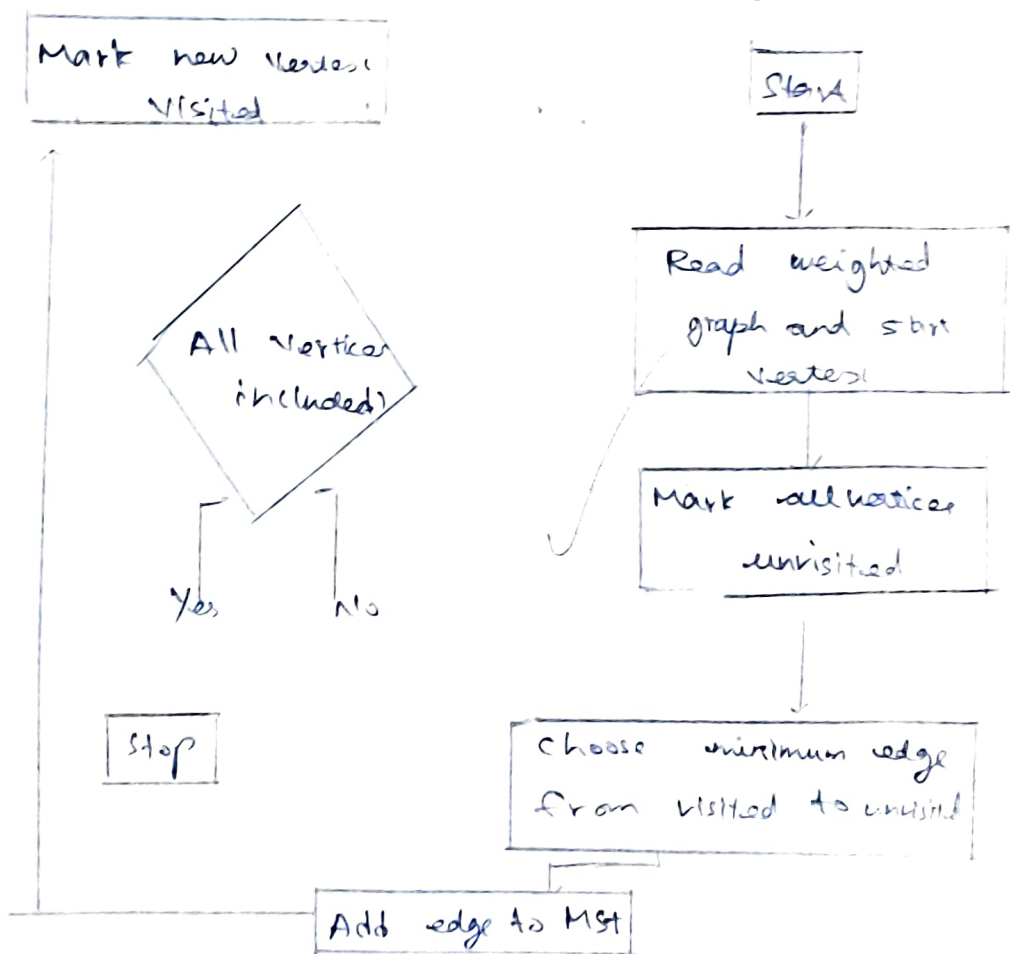
Enter source vertex: 0

Enter destination vertex: 4

Shortest distance = 3

Shortest path: 0 1 3 4

④ Convert the flowchart for Prim's algorithm into code



1. Algorithm

Pricked < 1000000

Pricked < 1000000

define MAX 20

int graph [MAX] [MAX];

int selected [MAX];

int n;

void print ()

{ int edgeCount = 0;

int totalWeight = 0;

/* start from vertex 0 */

selected [0] = 1;

printf ("nEdges in Minimum Spanning Tree (n)");

while (edgeCount < n-1)

{ int min = INT_MAX;

int x = -1;

int y = -1;

/* Find minimum edge */

for (int i = 0; i < n; i++)

{ if (selected [i] == 0)

{ for (int j = 0; j < n; j++)

{ if (selected [j] == 0 &&

graph [i][j] != 0 &&

graph [i][j] < min)

```
min = graph[i][j],
```

```
x = i,
```

```
y = j,
```

```
}
```

```
10 Check disconnected graph /
```

```
if (x == -1 || y == -1)
```

```
{ printf("Graph is disconnected \n");
```

```
return;
```

```
}
```

```
10 Add edge to MST /
```

```
printf("%d = %d + %d \n", x, y, min);
```

```
totalWeight += min;
```

```
selected [y] = 1;
```

```
edgeCount++;
```

```
}
```

```
printf("\n Total MST Weight = %d \n", totalWeight);
```

```
3
```

```
int main()
```

```
{ printf("Enter number of vertices: ");
```

```
scanf("%d", &n);
```

```
}
```

Prims (1)

returns

)

Output

Enter number of vertices: 5

Enter weighted adjacency matrix:

0	2	0	6	0
2	0	3	8	5
0	3	0	0	7
6	8	0	0	9
0	5	7	9	0

Edges in Minimum Spanning Tree:

0 → 1 : 2

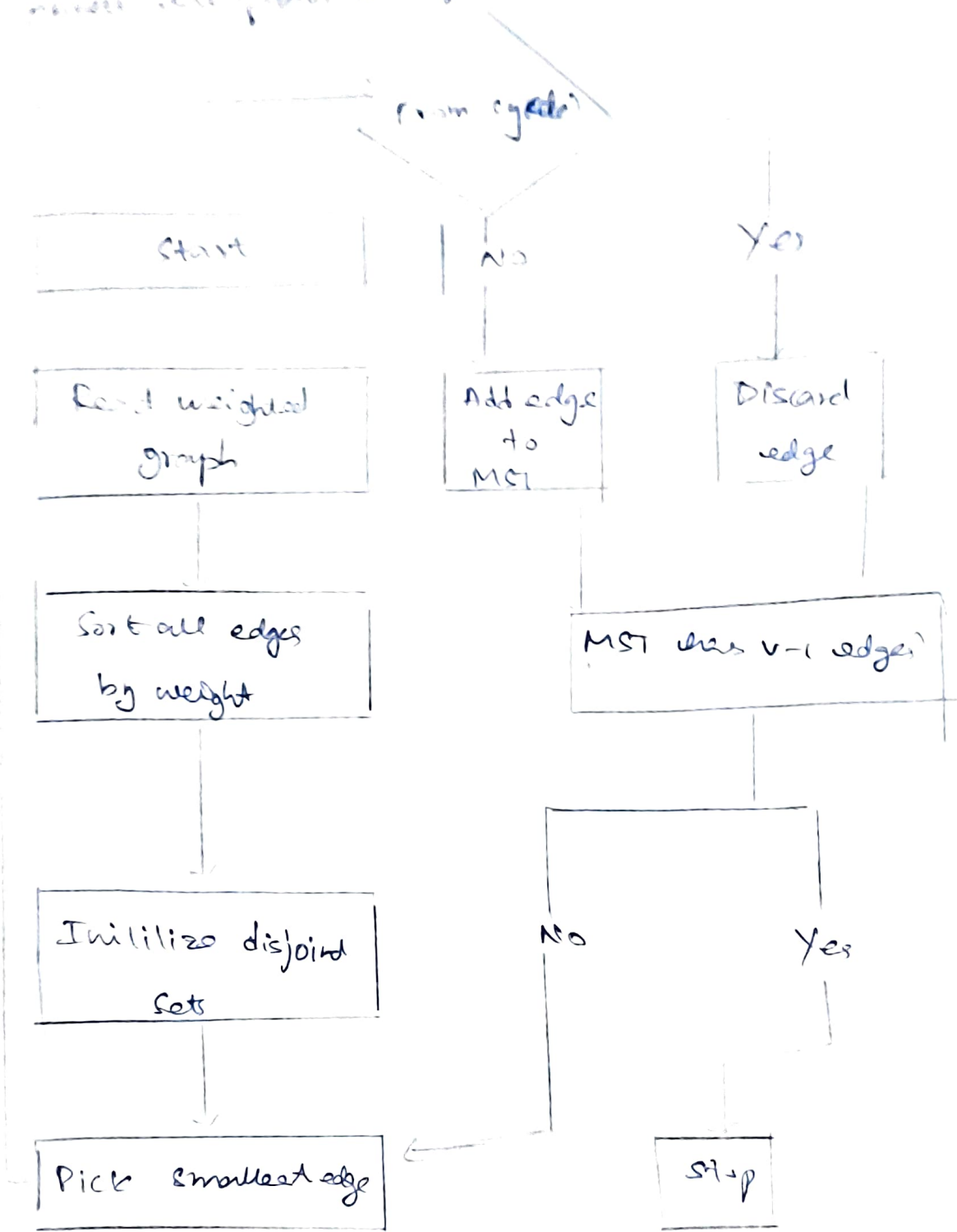
1 → 2 : 3

1 → 4 : 5

0 → 3 : 6

Total MST Weight = 16

minimize the span of the tree for a given set of points



Kruskal's Algorithm

```
# include <stdio.h>
```

```
# define MAX 100
```

```
typedef struct
```

```
{ int source;  
  int destination;  
  int weight;  
} Edge;
```

```
Edge edges[MAX];
```

```
int parent[MAX];
```

```
int n;
```

```
int edge count;
```

```
/* Find the parent of a vertex */
```

```
int find (int vertex)
```

```
{ if (parent[vertex] == vertex  
    return vertex;
```

```
    return parent[vertex] = find (parent[vertex]);
```

```
}
```

```
/* join two sets */
```

```
void union set (int a, int b)
```

```
{ int rootA = find (a);
```

```
  int rootB = find (b);
```

```
  parent[rootB] = rootA;
```

```
}
```

if an edge according to weight.

void sortEdges()

{
 sort(edges.begin(), edges.end(), less())

for (int i = 0; i < edges.size(); i++)

{ if (edges[i].weight > edges[i+1].weight)

{ swap(edges[i], edges[i+1])

edges[i] = edges[i+1];

edges[i+1] = temp;

}

kruskal's Algorithm :

void kruskal()

{ int mstEdge = 0;

int totalWeight = 0;

// sort edges

sortEdges();

// Utilize parent

for (int i = 0; i < edges.size(); i++)

{ parent[i] = i;

}

print("In Edges In minimum Spanning Tree")

return edge;

int find(int i)

{ if (edges[i] != null) return find(edges[i]);

return i;

int rootU = find(u);

int rootV = find(v);

if (rootU != rootV)

{ // Check whether adding edge creates cycle

{ if (rootU != rootV)

{ printf("id = %d %d %d\n", u, v, edges[i].weight);

totalWeight += edges[i].weight;

unionSet(rootU, rootV);

mstEdges++;

}

if (mstEdges != n-1)

{ printf("\n Graph is disconnected");

}

else

{ printf("\n Total MST weight = %d\n", totalWeight);

}

int main()

{ printf("Enter number of vertices:");

scanf("%d", &n);

```

Print("Enter number of edges: ");
scanf("%d", &edgeCount);
Print("Enter edges (source destination weight) \n");
for (int i = 0; i < edgeCount; i++)
{
    scanf("%d %d %d", &edge[i].source,
            &edge[i].destination,
            &edge[i].weight);
}

kruskal();
return 0;
}

```

Output

Enter number of vertices: 5

Enter number of edges: 7

Enter edges (source destination weight):

0 1 2

0 3 6

1 2 3

1 3 8

1 4 5

2 4 7

3 4 9

Edges in Minimum Spanning Tree:

0 → 1 : 2

1 → 2 : 3

1 → 4 : 5

0 → 3 : 6

Total Minimum Spanning Tree Weight = 16